

12

Régularisation

Automne 2026

Techniques d'apprentissage automatique — 420-C74-SF — gr. 10017
Spécialiste en solutions d'intelligence artificielle — M. Swawola, M.Sc.

NOUS ÉCLAIRON.S.
VOUS BRILLEZ.

FORMATION CONTINUE
ET SERVICES AUX ENTREPRISES



Sommaire

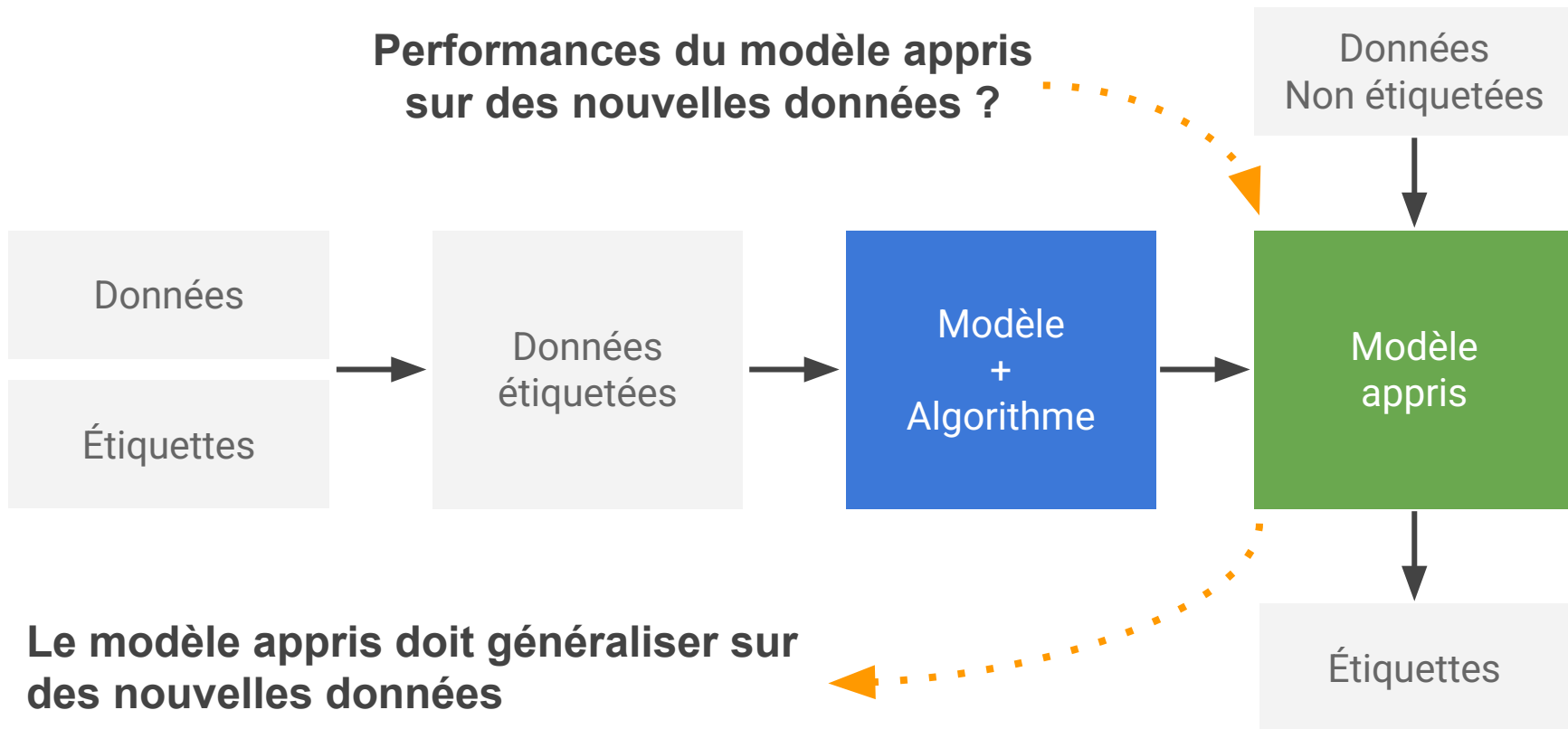
1. Introduction à la régularisation
2. Régularisation de Tikhonov
3. Lasso
4. Elastic net
5. Choix du paramètre de régularisation
6. Régression logistique et régularisation
7. Lectures et références

Sommaire

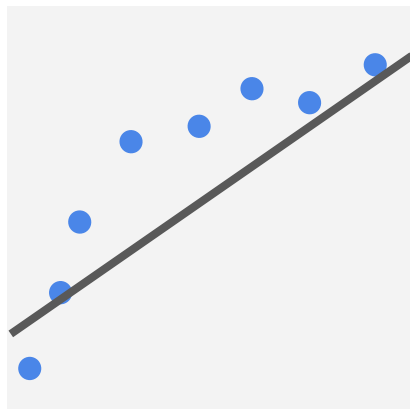
1. Introduction à la régularisation
2. Régularisation de Tikhonov
3. Lasso
4. Elastic net
5. Choix du paramètre de régularisation
6. Régression logistique et régularisation
7. Lectures et références

Généralisation (rappels)

**Performances du modèle appris
sur des nouvelles données ?**



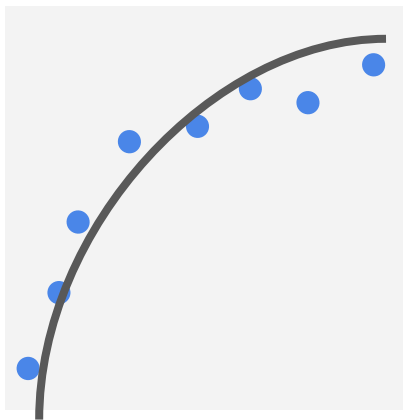
Généralisation en régression



$$\theta_0 + \theta_1 x$$

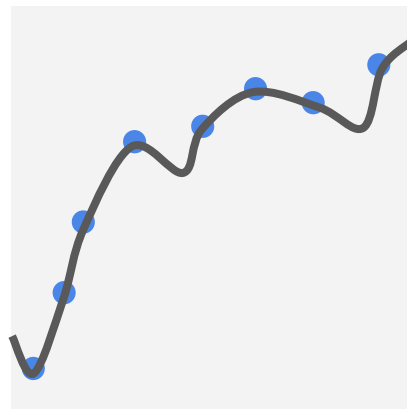
**Sous-apprentissage
(underfitting)**

Biais élevé



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

**Compromis
biais-variance**

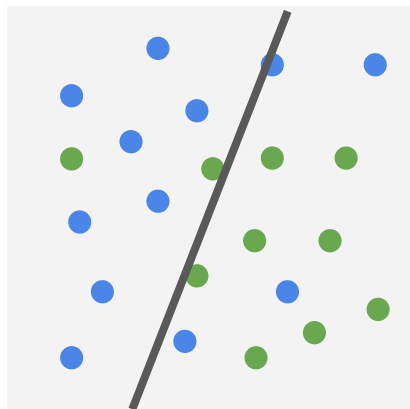


$$\theta_0 + \theta_1 x + \dots + \theta_5 x^5$$

**Surapprentissage
(overfitting)**

Variance élevée

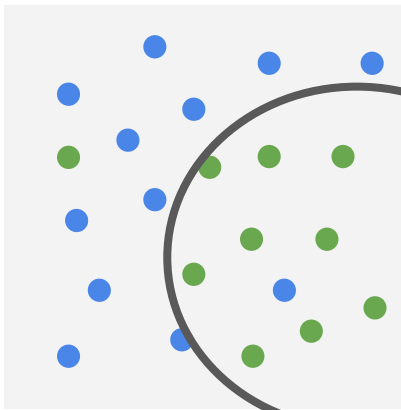
Généralisation en classification



$$g(\theta_0 + \theta_1 x)$$

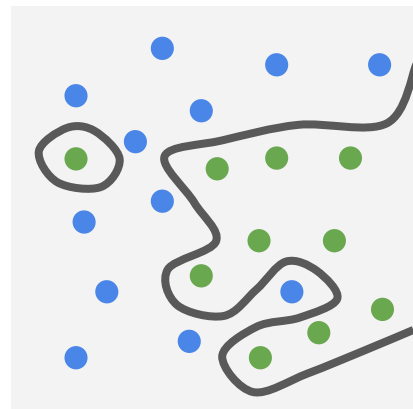
**Sous-apprentissage
(underfitting)**

Biais élevé



$$g(\theta_0 + \theta_1 x + \theta_2 x^2)$$

**Compromis
biais-variance**

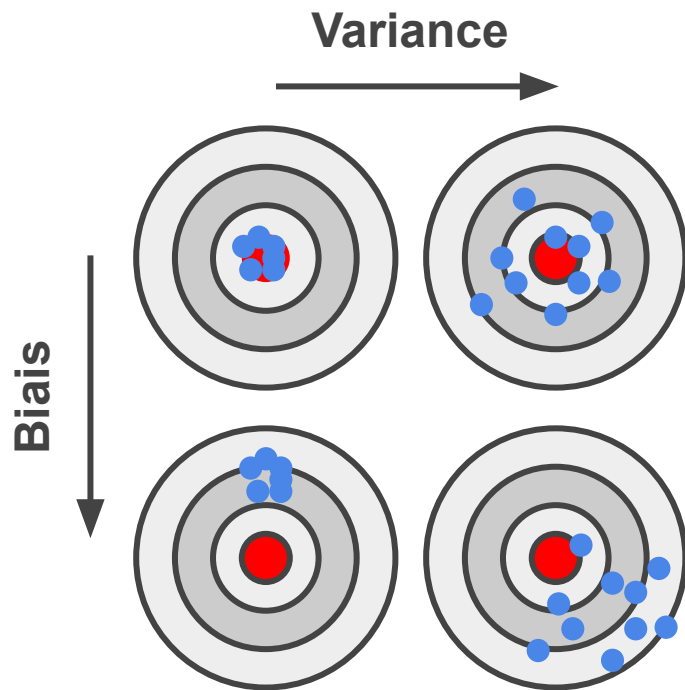


$$g(\theta_0 + \theta_1 x + \dots + \theta_5 x^5)$$

**Surapprentissage
(overfitting)**

Variance élevée

Biais vs Variance



Nombre de variables explicatives élevé

Exemple: prédiction du retard des vols

x_1 = aéroport de départ

x_2 = date et heure prévues du vol

x_3 = direction du vent

x_4 = température en °F

...

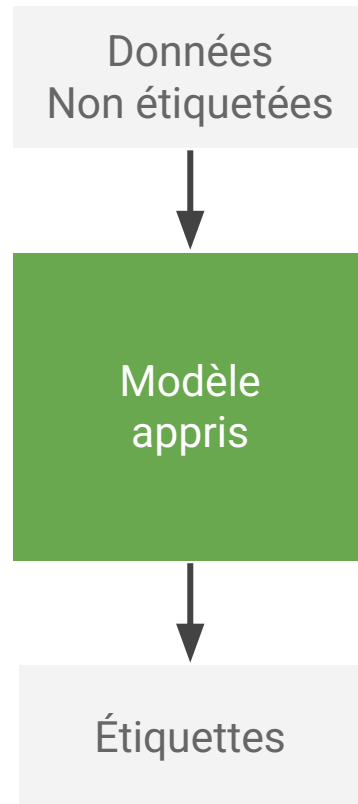
x_{100}



**Surapprentissage
(overfitting)**
Variance élevée

Surapprentissage

Très souvent, le **surapprentissage** est la première cause des mauvaises performances des modèles appris sur des nouvelles données



Quiz

Considérons un système de détection des intrusions dans un réseau informatique consistant en la classification des communications dans les catégories “intrusion” ou “normale”. Quelle situation correspond à un cas de surapprentissage ?

- A. Le système, basé sur un modèle affichant 89% de précision sur les données d'entraînement, détecte 90% des intrusions lors du test
- B. Le système affiche 65% de précision sur le jeu d'entraînement et montre les mêmes performances lors du test
- C. 70% des intrusions sont détectées lors du test pour un modèle affichant 90% de précision sur les données d'entraînement

Comment adresser le surapprentissage ?

Réduction du
nombre de
variables
explicatives

Réduction de
dimension

Régularisation

Comment adresser le surapprentissage ?

Réduction du
nombre de
variables
explicatives

Réduction de
dimension

Régularisation



Réalisée **manuellement ou par méthodes de sélection**, cette approche consiste à identifier un sous-ensemble des n variables explicatives et entraîner un modèle sur ce sous-ensemble

Comment adresser le surapprentissage ?

Réduction du
nombre de
variables
explicatives



Réduction de
dimension

Régularisation

Cette approche consiste à réaliser une projection des n variables explicatives sur un sous-espace de dimension $M < n$ et utiliser les M projections comme “nouvelles” variables explicatives

Comment adresser le surapprentissage ?

Réduction du
nombre de
variables
explicatives

Réduction de
dimension

Régularisation

Les n variables explicatives restent utilisées par le modèle, mais les valeurs des paramètres θ_j sont réduites, éventuellement jusqu'à zéro

Sommaire

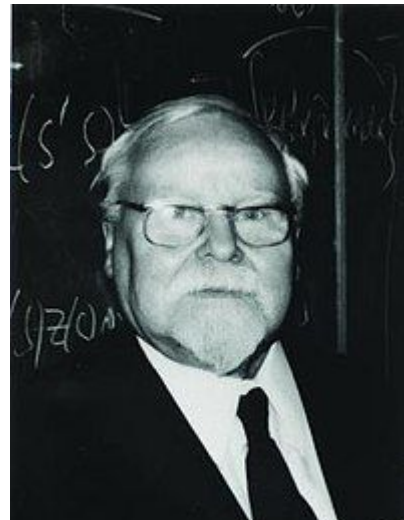
1. Introduction à la régularisation
2. Régularisation de Tikhonov
3. Lasso
4. Elastic net
5. Choix du paramètre de régularisation
6. Régression logistique et régularisation
7. Lectures et références

Origine et terminologie

→ Régularisation de **Tikhonov**

→ **Ridge** Regression

→ Régularisation **L2**



**Andrey
Nikolayevich
Tikhonov**

Régularisation de Tikhonov

RSS

$$J(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

Ajout d'une pénalité L2

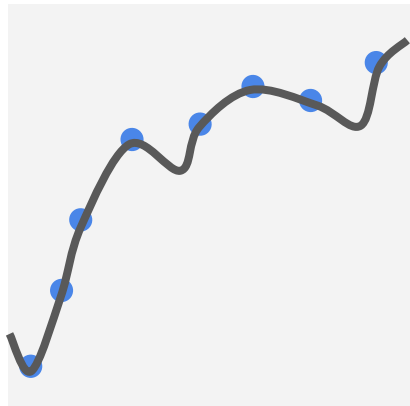
On cherche à minimiser $J(\theta)$

$\lambda \rightarrow$ paramètre de régularisation

→ Réduction des θ

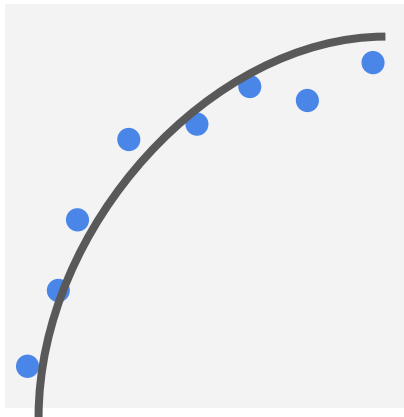
Paramètre de régularisation λ

$$J(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

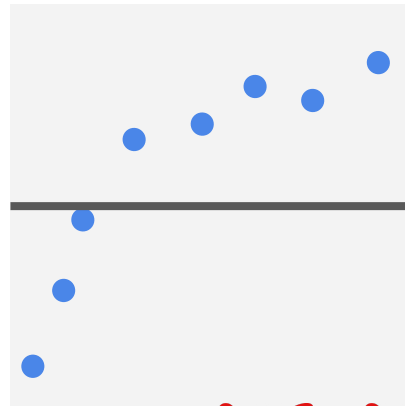
$$\theta_0 + \theta_1 x + \dots + \theta_5 x^5$$

$$\lambda = 0$$



$$\theta_0 + \theta_1 x + \dots + \theta_5 x^5$$

$$\lambda = 1000$$

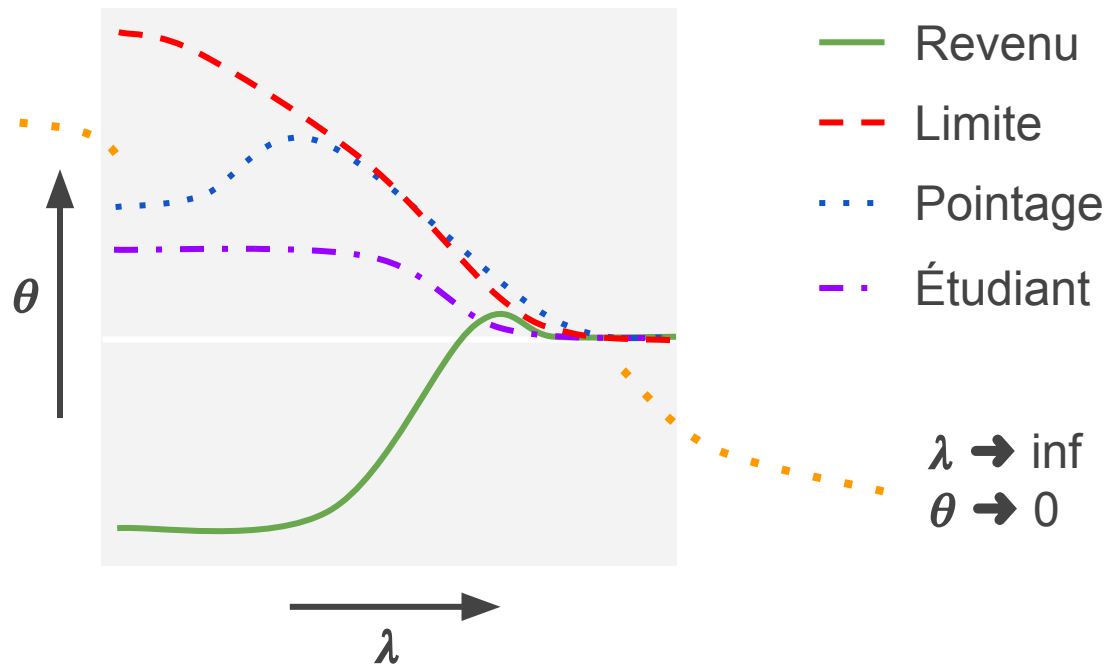


$$\theta_0 + \theta_1 x + \dots + \theta_5 x^5$$

$$\lambda = 10^{10}$$

Exemple - Jeu de données "Credit"

$\lambda = 0$
équivalent au
modèle sans
régularisation



Descente de gradient (optionnel)

Répéter jusqu'à convergence {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \times x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \times x_j^{(i)} - \frac{\lambda}{m} \theta_j \right]$$

}


$$\theta_j := \theta_j \underbrace{\left(1 - \alpha \frac{\lambda}{m}\right)}_{< 1} - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \times x_j^{(i)}$$

Question

En régression linéaire avec régularisation de Tikhonov, que se passe-t-il lorsque la valeur de λ est démesurément grande (par exemple 10^{10}) ?

$$J(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

- A. Il s'agit d'une situation de sous-apprentissage
- B. Tout fonctionne normalement car les grandes valeurs de λ ne peuvent pas nuire aux performances
- C. L'algorithme échoue à éviter le surapprentissage
- D. La descente de gradient diverge

Équation normale (optionnel)

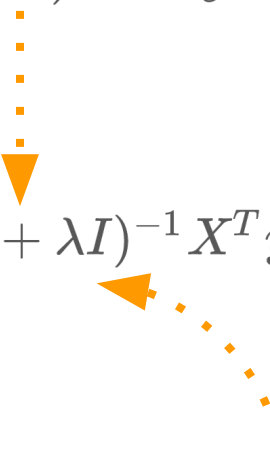
Sans régularisation :

$$\theta = (X^T X)^{-1} X^T y$$

Avec régularisation :

$$\theta = (X^T X + \lambda I)^{-1} X^T y$$

Si $\lambda > 0$, alors $X^T X + \lambda I$ est **invertible**


$$\begin{bmatrix} 0 & & & \\ & 1 & & 0 \\ & & \ddots & \\ 0 & & & 1 \\ & & & & 1 \end{bmatrix}$$

Question

- Considérons l'équation normale incluant le terme de régularisation. Quelle est la dimension de la matrice I ?

$$\theta = (X^T X + \lambda I)^{-1} X^T y$$

- A. $(n+1) \times (n+1)$
- B. $m \times n$
- C. $n \times 1$
- D. $m \times 1$

Ridge (scikit-learn 0.22.2)

Valeurs du paramètre de régularisation λ

```
class sklearn.linear_model.Ridge(alpha=1.0, fit_intercept=True,  
    normalize=False, copy_X=True, max_iter=None, tol=0.001, solver='auto',  
    random_state=None)
```

Algorithme itératif

Sommaire

1. Introduction à la régularisation
2. Régularisation de Tikhonov
3. **Lasso**
4. Elastic net
5. Choix du paramètre de régularisation
6. Régression logistique et régularisation
7. Lectures et références

Origine et terminologie

→ Lasso ou **LASSO**

Least Absolute Shrinkage and **Selection** Operator

→ Régularisation **L1**

→ Popularisé par ce monsieur



**Robert
Tibshirani**

Lasso

Ajout d'une pénalité L1

$$J(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |\theta_j| \right]$$

On cherche à minimiser $J(\theta)$

$\lambda \rightarrow$ paramètre de régularisation

→ Réduction, voir annulation des θ

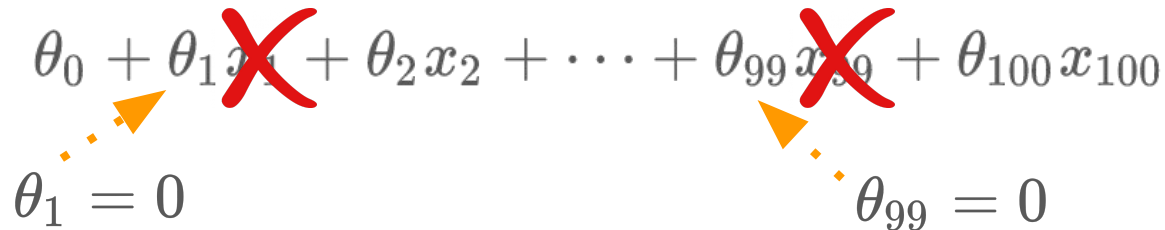
Lasso

Comme pour la régression de Tikhonov, le lasso réduit les paramètres θ

De plus, la pénalité L1 peut avoir pour effet de forcer à **exactement zéro** certains coefficients lorsque λ est suffisamment grand

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{99} x_{99} + \theta_{100} x_{100}$$

$\theta_1 = 0$ $\theta_{99} = 0$



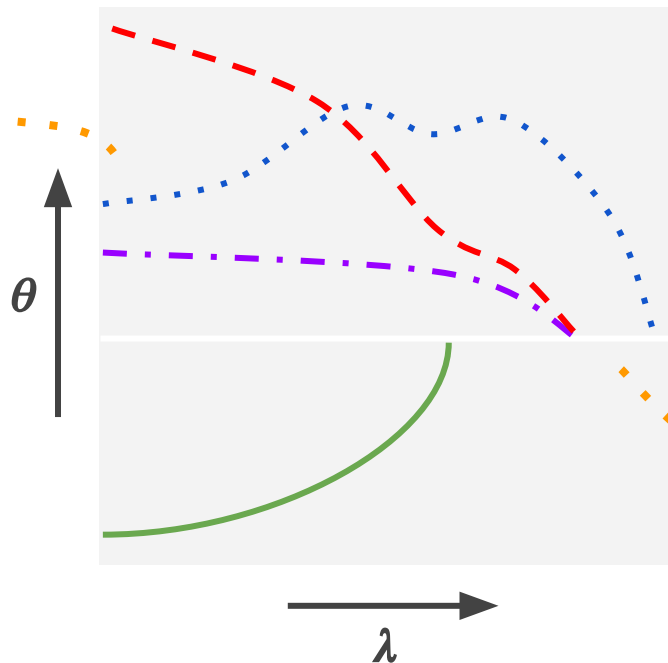
→ Sélection de variables (sparse models)

→ Importance de sélection de la valeur de λ

Exemple

Jeu de données "Credit"

$\lambda = 0$
équivalent au
modèle sans
régularisation



— Revenu
- - Limite
... Pointage
- . Étudiant

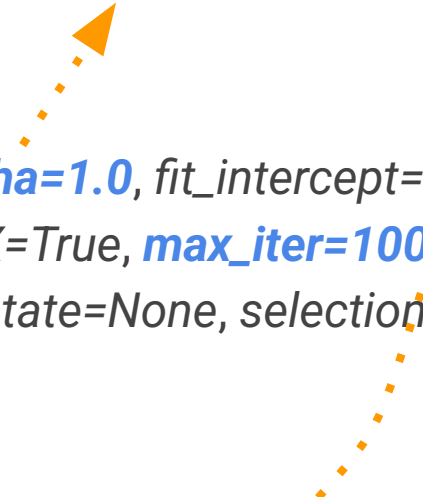
$\lambda \rightarrow \inf$
 $\theta = 0$ (sparse)

Lasso

(scikit-learn 0.22.2)

Valeurs du paramètre de régularisation λ

```
class sklearn.linear_model.Lasso(alpha=1.0, fit_intercept=True,  
    normalize=False, precompute=False, copy_X=True, max_iter=1000, tol=0.0001,  
    warm_start=False, positive=False, random_state=None, selection='cyclic')
```



Algorithme utilisé: **coordinate descent**

Sommaire

1. Introduction à la régularisation
2. Régularisation de Tikhonov
3. Lasso
4. Elastic net
5. Choix du paramètre de régularisation
6. Régression logistique et régularisation
7. Lectures et références

Elastic net

Limitations Lasso: $n > m$ et groupes de variables **hautement corrélées**

$$J(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda_1 \sum_{j=1}^n |\theta_j| + \lambda_2 \sum_{j=1}^n \theta_j^2 \right]$$

Pénalité L1 (Lasso)

Pénalité L2 (Tikhonov)

→ Combinaison linéaire des régularisation de Tikhonov et Lasso

ElasticNet

(scikit-learn 0.22.2)

```
class sklearn.linear_model.ElasticNet(alpha=1.0, l1_ratio=0.5,  
fit_intercept=True, normalize=False, precompute=False, max_iter=1000,  
copy_X=True, tol=0.0001, warm_start=False, positive=False, random_state=None,  
selection='cyclic')
```

$$\lambda_1 + \lambda_2$$



$$\frac{\lambda_1}{\lambda_1 + \lambda_2}$$

Sommaire

1. Introduction à la régularisation
2. Régularisation de Tikhonov
3. Lasso
4. Elastic net
5. Choix du paramètre de régularisation
6. Régression logistique et régularisation
7. Lectures et références

Choix de λ

λ est sélectionné par validation croisée

- Utilisation d'une grille de valeurs de λ
- Évaluation de l'erreur de validation pour chaque valeur de λ
- Sélection de λ pour laquelle l'erreur de validation est la plus faible
- Le modèle est ré-entraîné sur l'ensemble des données d'entraînement avec la valeur de λ sélectionnée



λ	MSE
0	125
10^2	76
10^3	92
10^4	110

RidgeCV

(scikit-learn 0.22.2)

Valeurs du paramètre de régularisation λ

```
class sklearn.linear_model.RidgeCV(  
    alphas=(0.1, 1.0, 10.0), fit_intercept=True, normalize=False, scoring=None,  
    cv=None, gcv_mode=None, store_cv_values=False)
```

LOOCV par défaut

Attribut **cv_values_**

LassoCV

(scikit-learn 0.22.2)

Valeurs du paramètre de régularisation λ

```
class sklearn.linear_model.LassoCV(  
    eps=0.001, n_alphas=100, alphas=None, fit_intercept=True, normalize=False,  
    precompute='auto', max_iter=1000, tol=0.0001, copy_X=True, cv=None,  
    verbose=False, n_jobs=None, positive=False, random_state=None,  
    selection='cyclic')
```

5-plis par défaut

ElasticNetCV

(scikit-learn 0.22.2)

Valeurs du paramètre de régularisation λ

```
class sklearn.linear_model.ElasticNetCV(l1_ratio=0.5,  
    eps=0.001, n_alphas=100, alphas=None, fit_intercept=True, normalize=False,  
    precompute='auto', max_iter=1000, tol=0.0001, cv=None, copy_X=True,  
    verbose=0, n_jobs=None, positive=False, random_state=None, selection='cyclic')
```

5-plis par défaut

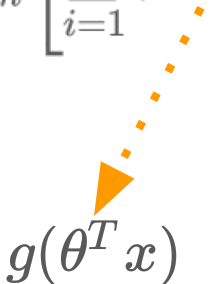
Sommaire

1. Introduction à la régularisation
2. Régularisation de Tikhonov
3. Lasso
4. Elastic net
5. Choix du paramètre de régularisation
6. Régression logistique et régularisation
7. Lectures et références

Régression logistique et régularisation

➔ Les mêmes principes qu'en régression linéaire s'appliquent !

$$J(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda_1 \sum_{j=1}^n |\theta_j| + \lambda_2 \sum_{j=1}^n \theta_j^2 \right]$$



$$g(\theta^T x)$$

Descente de gradient (Ridge) (optionnel)

Répéter jusqu'à convergence {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \times x_0^{(i)}$$

$$\theta_j := \theta_j (1 - \alpha \frac{\lambda}{m}) - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \times x_j^{(i)}$$

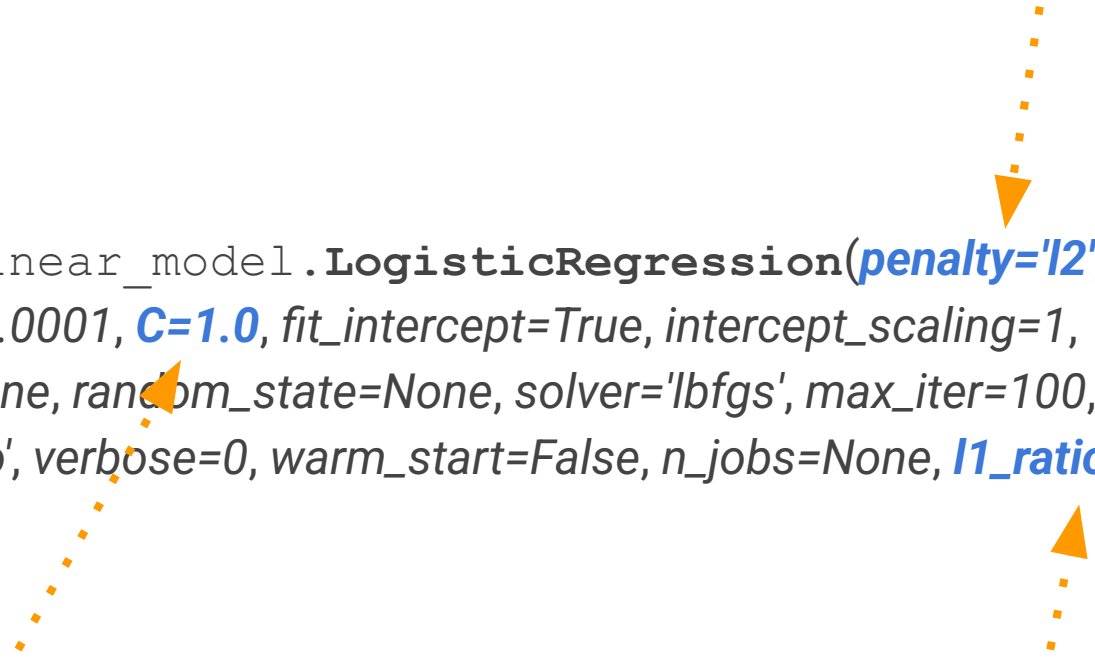
}


$$g(\theta^T x)$$

LogisticRegression (scikit-learn 0.22.2)

l1, l2, elasticnet ou none

```
class sklearn.linear_model.LogisticRegression(penalty='l2',  
    dual=False, tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1,  
    class_weight=None, random_state=None, solver='lbfgs', max_iter=100,  
    multi_class='auto', verbose=0, warm_start=False, n_jobs=None, l1_ratio=None)
```



Paramètre de régularisation (inverse)

$$C = 1/\lambda$$

Ratio pénalités (elasticnet)

LogisticRegressionCV (scikit-learn 0.22.2)

Valeurs du paramètre C (inverse λ)

```
class sklearn.linear_model.LogisticRegressionCV(Cs=10,  
fit_intercept=True, cv=None, dual=False, penalty='l2', scoring=None, solver='lbfgs',  
tol=0.0001, max_iter=100, class_weight=None, n_jobs=None, verbose=0,  
refit=True, intercept_scaling=1.0, multi_class='auto', random_state=None,  
l1_ratios=None)
```

5-plis stratifié par défaut

RidgeClassifier

(scikit-learn 0.22.2)

Classificateur basé sur une régression Ridge

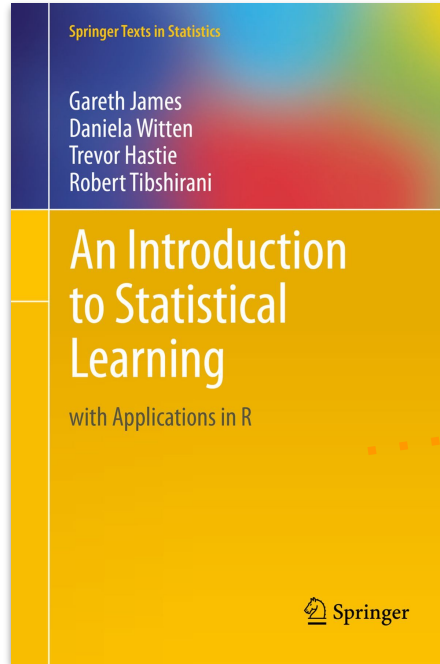
```
class sklearn.linear_model.RidgeClassifier(alpha=1.0,  
    fit_intercept=True, normalize=False, copy_X=True, max_iter=None, tol=0.001,  
    class_weight=None, solver='auto', random_state=None)
```

→ **RidgeClassifier** peut être beaucoup plus rapide que **LogisticRegression** lorsque le nombre de classes est très élevé

→ Il existe aussi **RidgeClassifierCV**

Sommaire

1. Introduction à la régularisation
2. Régularisation de Tikhonov
3. Lasso
4. Elastic net
5. Choix du paramètre de régularisation
6. Régression logistique et régularisation
7. Lectures et références



6.2 Shrinkage Methods
p. 214-228

Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, “An Introduction to Statistical Learning with Applications in R”

Lectures et références

- [1] Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, **“An Introduction to Statistical Learning with Applications in R”**
- [2] CS229: Machine Learning - Stanford University
- [3] Cours “Évaluation des modèles appris”, Patrick Dallaire, CRDM